

Market Basket Analysis with Python - eBay

Project overview:

You are hired as a Customer Insights Analyst for eBay, a leading global e-commerce platform. With millions of customers making purchases across diverse categories, the leadership team wants to understand customer purchasing behavior, product affinities, and engagement trends to enhance personalized shopping experiences and optimize inventory and marketing strategies.

The leadership is particularly interested in understanding:

- What products or categories are frequently purchased together, indicating cross-selling opportunities?
- How do customer purchasing patterns vary across demographics or regions?
- Can we segment customers based on buying behavior to target personalized promotions?
- Which products drive repeat purchases or high customer loyalty?

Objective:

Build a comprehensive Customer Purchasing Behavior and Product Recommendation Report using Python for data analysis. This report should clean and consolidate eBay's transaction and customer data, apply techniques such as Market Basket Analysis, clustering, and association rule mining, and provide actionable insights for personalized product recommendations, promotional strategies, and inventory optimization.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import LabelEncoder
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score
```

```
df = pd.read_csv("eBay1.csv")
df.head(10)
```

	Timestamp	age	Gender	Purchase_Frequency	Purchase_Categories	Personalized_Recommendation_Frequency	Browsing_Frequency	Product_Search_Method	Search
0	2023/06/07 11:44:55 AM GMT+5:30	32	Prefer not to say	Multiple times a week	Groceries and Gourmet Food;Home and Kitchen		Sometimes	Rarely	categories
1	2023/06/07 9:28:09 AM GMT+5:30	47	Female	Multiple times a week	Groceries and Gourmet Food;Beauty and Personal...		No	Few times a month	NaN
2	2023/06/05 10:09:03 PM GMT+5:30	50	Female	Once a month	Groceries and Gourmet Food;Beauty and Personal...		Sometimes	Multiple times a day	categories
3	2023/06/07 5:58:12 PM GMT+5:30	6	Others	Once a month	Groceries and Gourmet Food;Beauty and Personal...		No	Rarely	others
4	2023/06/07 11:46:52 AM GMT+5:30	61	Male	Once a week	Groceries and Gourmet Food;Clothing and Fashion		Yes	Rarely	NaN
5	2023/06/08 5:49:56 PM GMT+5:30	14	Prefer not to say	Few times a month	Clothing and Fashion		Sometimes	Few times a month	NaN
6	2023/06/09 2:36:09 PM GMT+5:30	31	Female	Once a week	Clothing and Fashion;others		Yes	Few times a month	categories

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 800 entries, 0 to 799
Data columns (total 24 columns):
 #   Column                                Non-Null Count  Dtype
---  -
 0   Timestamp                            800 non-null    object
 1   age                                  800 non-null    int64
 2   Gender                              800 non-null    object
 3   Purchase_Frequency                  800 non-null    object
 4   Purchase_Categories                 800 non-null    object
 5   Personalized_Recommendation_Frequency 800 non-null    object
 6   Browsing_Frequency                 800 non-null    object
 7   Product_Search_Method              639 non-null    object
 8   Search_Result_Exploration           800 non-null    object
 9   Customer_Reviews_Importance         800 non-null    int64
10   Add_to_Cart_Browsing               800 non-null    object
11   Cart_Completion_Frequency           800 non-null    object
12   Cart_Abandonment_Factors            800 non-null    object
13   Saveforlater_Frequency             800 non-null    object
14   Review_Left                        800 non-null    object
15   Review_Reliability                 800 non-null    object
16   Review_Helpfulness                 800 non-null    object
17   Personalized_Recommendation_Frequency 800 non-null    int64
18   Recommendation_Helpfulness          800 non-null    object
19   Rating_Accuracy                    800 non-null    int64
20   Shopping_Satisfaction               800 non-null    int64
21   Service_Appreciation               800 non-null    object
22   Improvement_Areas                  800 non-null    object
23   transaction                        800 non-null    int64
dtypes: int64(6), object(18)
memory usage: 150.1+ KB
```

Task 1: Data Cleaning and Preparation

- Remove duplicate or inconsistent survey responses.
- Standardize categorical/numerical entries (e.g., frequency levels, gender, recommendation responses).
- Handle missing values and inconsistent formats in Product_Search_Method and other fields.
- Rename duplicate or misformatted columns (e.g., remove trailing spaces in Rating_Accuracy).
- Convert numerical rating columns (e.g., Customer_Reviews_Importance, Shopping_Satisfaction) to appropriate numeric types for analysis.

```
df_clean = df.drop_duplicates()
```

```
df.head(10)
```

```
## there are no null values and duplicate values
```

	Timestamp	age	Gender	Purchase_Frequency	Purchase_Categories	Personalized_Recommendation_Frequency	Browsing_Frequency	Product_Search_Method	Search
0	2023/06/07 11:44:55 AM GMT+5:30	32	Prefer not to say	Multiple times a week	Groceries and Gourmet Food:Home and Kitchen	Sometimes	Rarely	categories	
1	2023/06/07 9:28:09 AM GMT+5:30	47	Female	Multiple times a week	Groceries and Gourmet Food:Beauty and Personal...	No	Few times a month	NaN	
2	2023/06/05 10:09:03 PM GMT+5:30	50	Female	Once a month	Groceries and Gourmet Food:Beauty and Personal...	Sometimes	Multiple times a day	categories	
3	2023/06/07 5:58:12 PM GMT+5:30	6	Others	Once a month	Groceries and Gourmet Food:Beauty and Personal...	No	Rarely	others	
4	2023/06/07 11:46:52 AM GMT+5:30	61	Male	Once a week	Groceries and Gourmet Food:Clothing and Fashion	Yes	Rarely	NaN	
5	2023/06/08 5:49:56 PM GMT+5:30	14	Prefer not to say	Few times a month	Clothing and Fashion	Sometimes	Few times a month	NaN	
6	2023/06/09 2:36:09 PM	31	Female	Once a week	Clothing and Fashion	Yes	Few times a month	categories	

```
## remove duplicate columns by index
```

```
df = df.drop(df.columns[5], axis=1)
```

```
df.head()
```

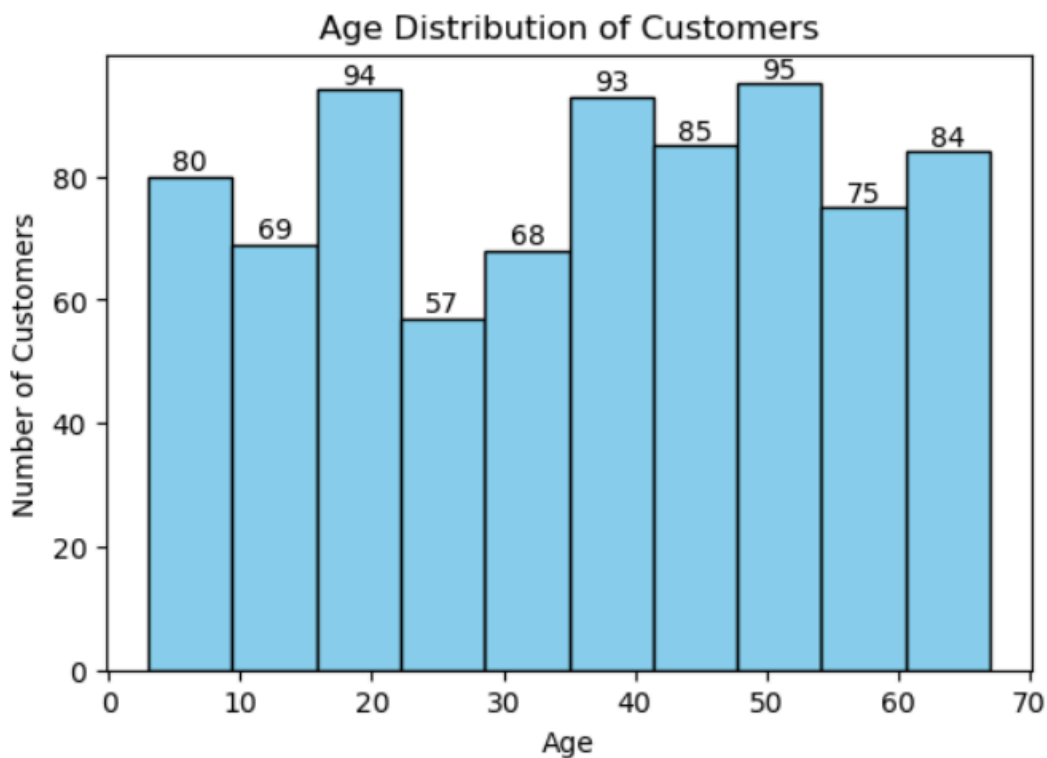
	Timestamp	age	Gender	Purchase_Frequency	Purchase_Categories	Browsing_Frequency	Product_Search_Method	Search_Result_Exploration	Customer_Reviews_Importance
0	2023/06/07 11:44:55 AM GMT+5:30	32	Prefer not to say	Multiple times a week	Groceries and Gourmet Food:Home and Kitchen	Rarely	categories	Multiple pages	
1	2023/06/07 9:28:09 AM GMT+5:30	47	Female	Multiple times a week	Groceries and Gourmet Food:Beauty and Personal...	Few times a month	NaN	First page	
2	2023/06/05 10:09:03 PM GMT+5:30	50	Female	Once a month	Groceries and Gourmet Food:Beauty and Personal...	Multiple times a day	categories	Multiple pages	
3	2023/06/07 5:58:12 PM GMT+5:30	6	Others	Once a month	Groceries and Gourmet Food:Beauty and Personal...	Rarely	others	Multiple pages	
4	2023/06/07 11:46:52 AM GMT+5:30	61	Male	Once a week	Groceries and Gourmet Food:Clothing and Fashion	Rarely	NaN	Multiple pages	

Task 2: Descriptive Behavior Analysis

Summarize customer demographics (age, gender distribution).

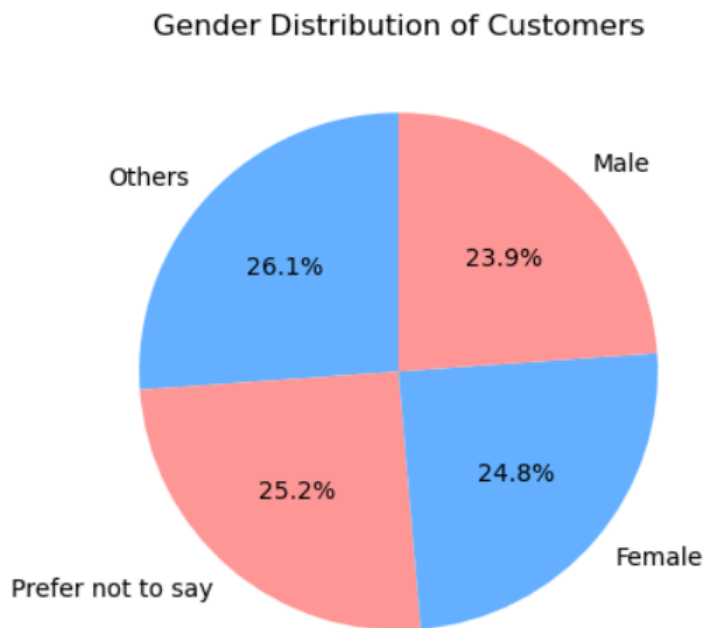
Age Distribution (Histogram)

```
plt.figure(figsize=(6,4))
counts, bins, patches = plt.hist(
    df["age"].dropna(),
    bins=10,
    color="skyblue",
    edgecolor="black"
)
for count, patch in zip(counts, patches):
    plt.text(
        patch.get_x() + patch.get_width() / 2,
        count,
        int(count),
        ha="center",
        va="bottom"
    )
plt.xlabel("Age")
plt.ylabel("Number of Customers")
plt.title("Age Distribution of Customers")
plt.show()
```



Gender Distribution pie chart

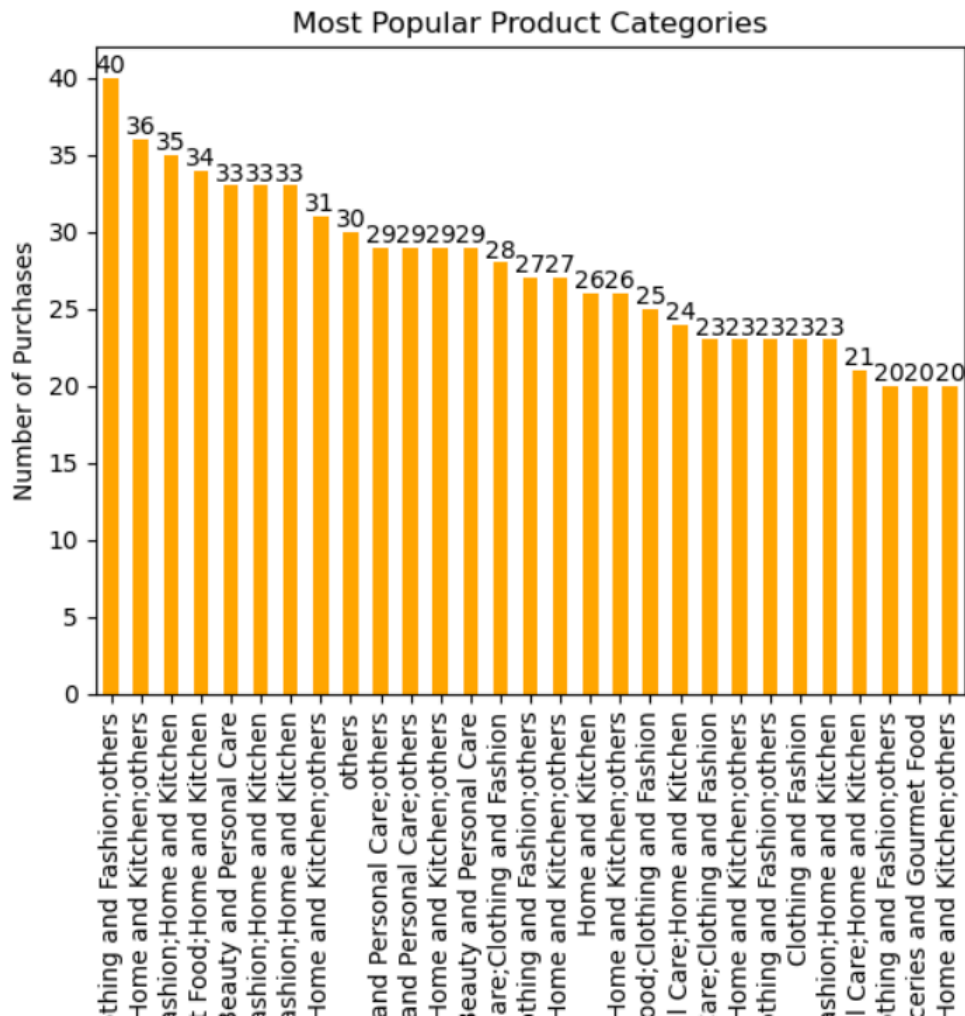
```
df["Gender"].value_counts().plot(
    kind="pie",
    autopct="%1.1f%%",
    startangle=90,
    colors=["#66b3ff", "#ff9999"]
)
plt.ylabel("")
plt.title("Gender Distribution of Customers")
plt.show()
```



Analyze overall purchase frequency and most popular product categories.

```
product_counts = df["Purchase_Categories"].value_counts()
product_counts
```

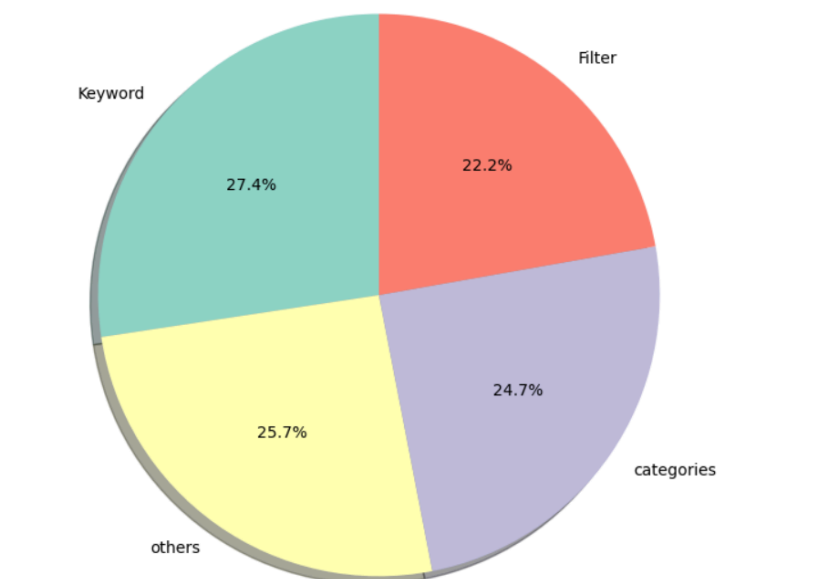
```
[26]: Purchase_Categories
Clothing and Fashion;others 40
Clothing and Fashion;Home and Kitchen;others 36
Groceries and Gourmet Food;Beauty and Personal Care;Clothing and Fashion;Home and Kitchen 35
Groceries and Gourmet Food;Home and Kitchen 34
Groceries and Gourmet Food;Beauty and Personal Care 33
Groceries and Gourmet Food;Clothing and Fashion;Home and Kitchen 33
Beauty and Personal Care;Clothing and Fashion;Home and Kitchen 33
Beauty and Personal Care;Clothing and Fashion;Home and Kitchen;others 31
others 30
Groceries and Gourmet Food;Beauty and Personal Care;others 29
Beauty and Personal Care;others 29
Beauty and Personal Care;Home and Kitchen;others 29
Beauty and Personal Care 29
Beauty and Personal Care;Clothing and Fashion 28
Beauty and Personal Care;Clothing and Fashion;others 27
Groceries and Gourmet Food;Beauty and Personal Care;Clothing and Fashion;Home and Kitchen;others 27
Home and Kitchen 26
Home and Kitchen;others 26
Groceries and Gourmet Food;Clothing and Fashion 25
Beauty and Personal Care;Home and Kitchen 24
Groceries and Gourmet Food;Beauty and Personal Care;Clothing and Fashion 23
Groceries and Gourmet Food;Clothing and Fashion;Home and Kitchen;others 23
Groceries and Gourmet Food;Clothing and Fashion;others 23
Clothing and Fashion 23
Clothing and Fashion;Home and Kitchen 23
Groceries and Gourmet Food;Beauty and Personal Care;Home and Kitchen 21
Groceries and Gourmet Food;Beauty and Personal Care;Clothing and Fashion;others 20
Groceries and Gourmet Food 20
Groceries and Gourmet Food;Home and Kitchen;others 20
Name: count, dtype: int64
```

Identify top browsing methods and most common cart abandonment factors.

```
df["Product_Search_Method"].value_counts().plot(
    kind="pie",
    autopct="%1.1f%%",
    startangle=90,
    figsize=(8,8),
    colors=sns.color_palette("Set3"),
    shadow=True
)
plt.ylabel("")
plt.title("Browsing Methods Share", fontsize=16)
plt.show()
```

Browsing Methods Share



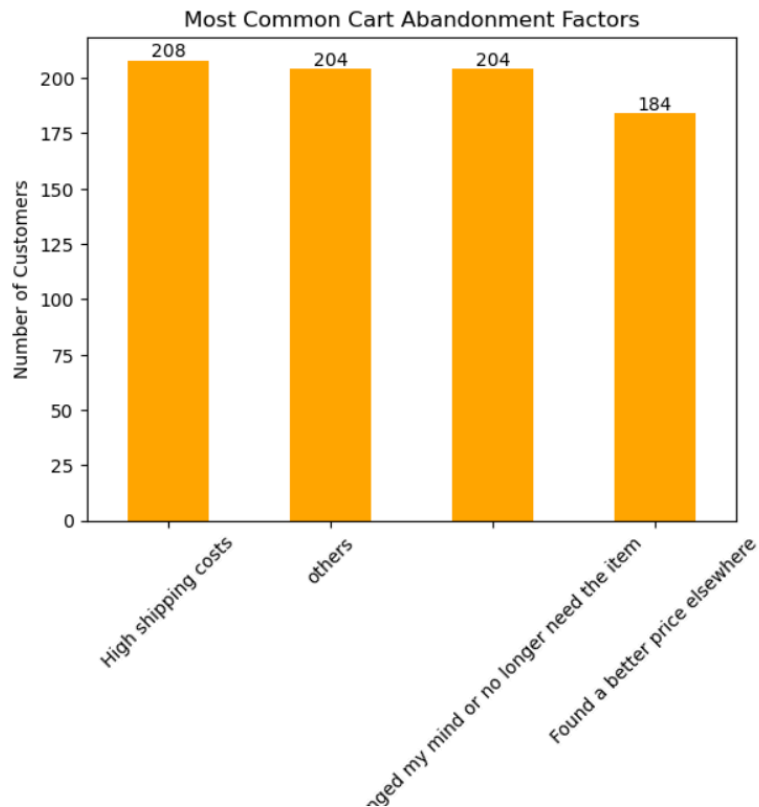
Visualization for Cart Abandonment

```
ax = df["Cart_Abandonment_Factors"].value_counts().plot(
    kind="bar",
    title="Most Common Cart Abandonment Factors",
    color="orange"
)
```

Add values on top of bars

```
for p in ax.patches:
    ax.annotate(int(p.get_height()),
                (p.get_x() + p.get_width()/2, p.get_height()),
                ha="center",
                va="bottom")
```

```
plt.xlabel("Cart_Abandonment_Factors")
plt.ylabel("Number of Customers")
plt.xticks(rotation=45)
plt.show()
```



Calculate mean and median satisfaction, recommendation helpfulness, and rating accuracy.

```
j> df[["Shopping_Satisfaction"]].agg(["mean", "median"])
```

```
j> Shopping_Satisfaction
```

mean	2.86625
median	3.00000

```
j> df["Recommendation_Helpfulness_numeric"] = df["Recommendation_Helpfulness"].map({"yes": 1, "no": 0})
j> df[["Recommendation_Helpfulness_numeric"]].agg(["mean", "median"])
```

```
j> Recommendation_Helpfulness_numeric
```

mean	NaN
median	NaN

```
j> df[["Rating_Accuracy "]].agg(["mean", "median"])
```

```
j> Rating_Accuracy
```

mean	3.08625
median	3.00000

```
j>
```

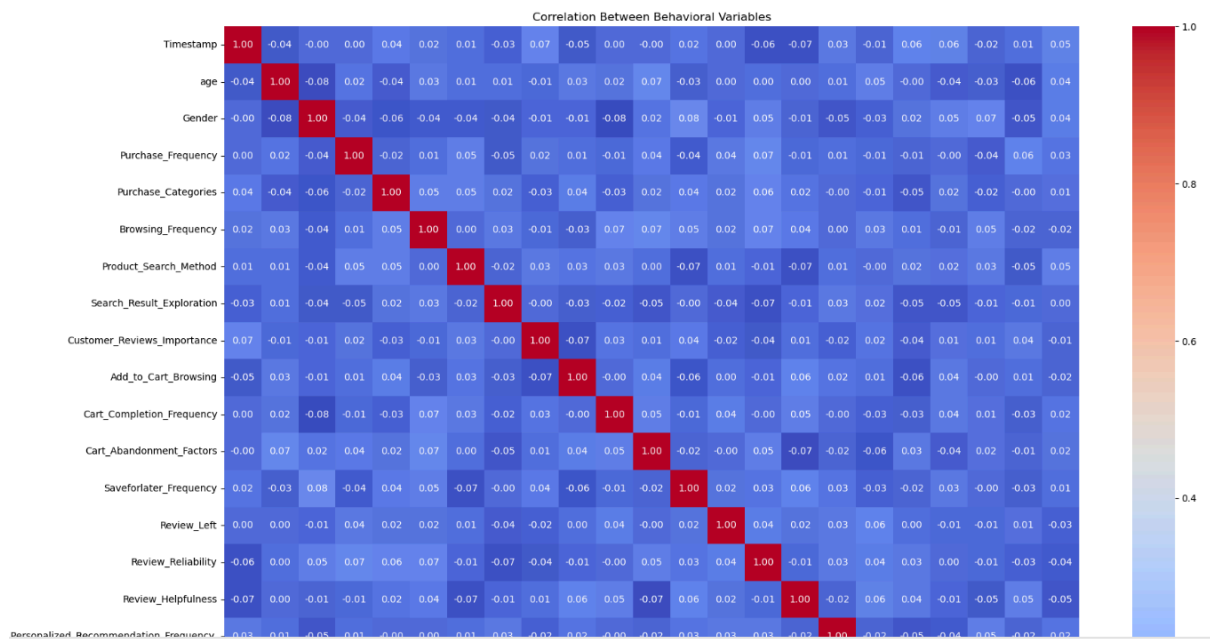
Generate summary statistics and visualizations for key behavioral variables.

Encoding Categorical Variables

```
categorical_cols = df.select_dtypes(include=["object"]).columns
encoder = LabelEncoder()
for col in categorical_cols:
    df[col] = encoder.fit_transform(df[col].astype(str))
```

Behavioral Correlation Heatmap

```
plt.figure(figsize=(20, 16))
corr = df[categorical_cols].corr()
sns.heatmap(
    corr,
    annot=True,
    cmap="coolwarm",
    fmt=".2f"
)
plt.title("Correlation Between Behavioral Variables")
plt.show()
```



Task 3: Customer Segmentation and Profiling

Segment customers based on purchase frequency and shopping satisfaction levels.

Select Only Required Columns

```
X = df[["Purchase_Frequency", "Shopping_Satisfaction"]]
```

Scale the Data

```
X_scaled = StandardScaler().fit_transform(X)
```

Apply K-Means

```
kmeans = KMeans(n_clusters=3, random_state=42)  
df["Segment"] = kmeans.fit_predict(X_scaled)
```

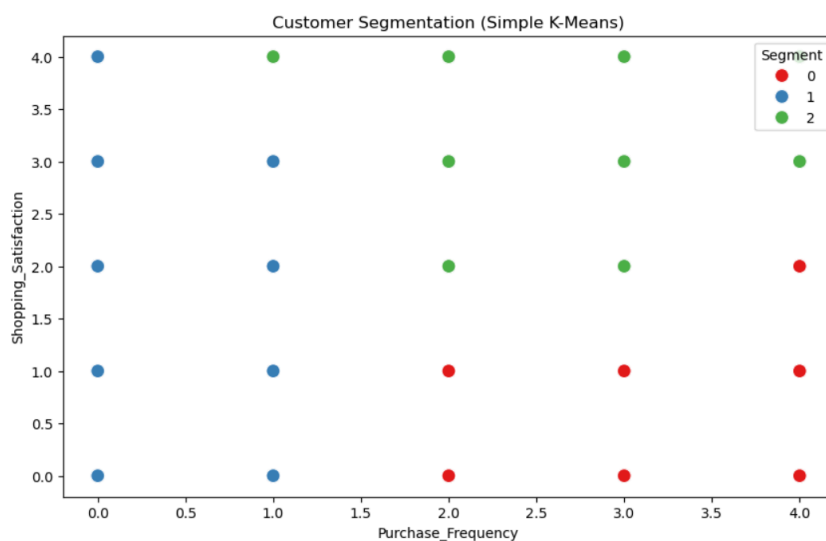
Visualization

```
plt.figure(figsize=(10, 6))
```

```
sns.scatterplot(  
    x="Purchase_Frequency",  
    y="Shopping_Satisfaction",  
    hue="Segment",  
    data=df,  
    palette="Set1",  
    s=100  
)
```

```
plt.title("Customer Segmentation (Simple K-Means)")
```

```
plt.show()
```



Segment 1 (blue) clusters at low purchase frequency but spans the full satisfaction scale, suggesting inconsistent experience among light shoppers.

Segment 0 (red) concentrates at higher purchase frequency with low–mid satisfaction, indicating frequent shoppers who are not consistently happy.

Segment 2 (green) appears at medium–high purchase frequency with generally higher satisfaction, representing a valuable core group that is both active and relatively satisfied.

```
df.groupby("Segment")[["Purchase_Frequency", "Shopping_Satisfaction"]].mean()
```

Purchase_Frequency Shopping_Satisfaction		
Segment		
0	3.160000	0.653333
1	0.446945	1.623794
2	2.613636	3.185606

Create profiles such as:

Frequent Buyers: High purchase frequency, high satisfaction.

Occasional Shoppers: Medium frequency, moderate satisfaction.

At-Risk Customers: Low satisfaction or frequent cart abandonment.

```
features = df[["Purchase_Frequency", "Shopping_Satisfaction", "Cart_Completion_Frequency"]]
```

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(features)
```

```
kmeans = KMeans(n_clusters=3, random_state=42)
df["Cluster"] = kmeans.fit_predict(X_scaled)
```

```
cluster_profile = df.groupby("Cluster")[["Purchase_Frequency", "Shopping_Satisfaction", "Cart_Completion_Frequency"]].mean()
```

cluster_profile

	Purchase_Frequency	Shopping_Satisfaction	Cart_Completion_Frequency
Cluster			
0	3.380328	1.865574	1.793443
1	0.830189	1.818868	0.864151
2	1.256522	1.921739	3.660870

segmented based on the clusters and profiles

```
def label_customer(row):
```

```
    if row["Purchase_Frequency"] > cluster_profile["Purchase_Frequency"].mean()
    and \
        row["Shopping_Satisfaction"] > cluster_profile["Shopping_Satisfaction"].mean():
        return "Frequent Buyers"
```

```
    elif row["Shopping_Satisfaction"] <
cluster_profile["Shopping_Satisfaction"].mean():
        return "At-Risk Customers"
```

```
    else:
        return "Occasional Shoppers"
```

```
df["Customer_Profile"] = df.apply(label_customer, axis=1)
```

Visualization

```
plt.figure(figsize=(12, 8))
```

```
sns.scatterplot(
    x="Purchase_Frequency",
    y="Shopping_Satisfaction",
    hue="Customer_Profile",
    data=df,
    palette="Set2",
    s=100
```

```
)
```

```
plt.title("Customer Profiles Based on Segmentation", fontsize=16)
```

```
plt.show()
```



```
df["Customer_Profile"].value_counts()
```

```
Customer_Profile
At-Risk Customers    351
Frequent Buyers      259
Occasional Shoppers  190
Name: count, dtype: int64
```

Analyze demographic or behavioral differences across these segments.

segment based on the behavioral columns

```
segment_summary = df.groupby("Customer_Profile")[
    "age",
    "Purchase_Frequency",
    "Shopping_Satisfaction",
    "Cart_Completion_Frequency",
    "Browsing_Frequency",
    "Personalized_Recommendation_Frequency "
].mean()
```

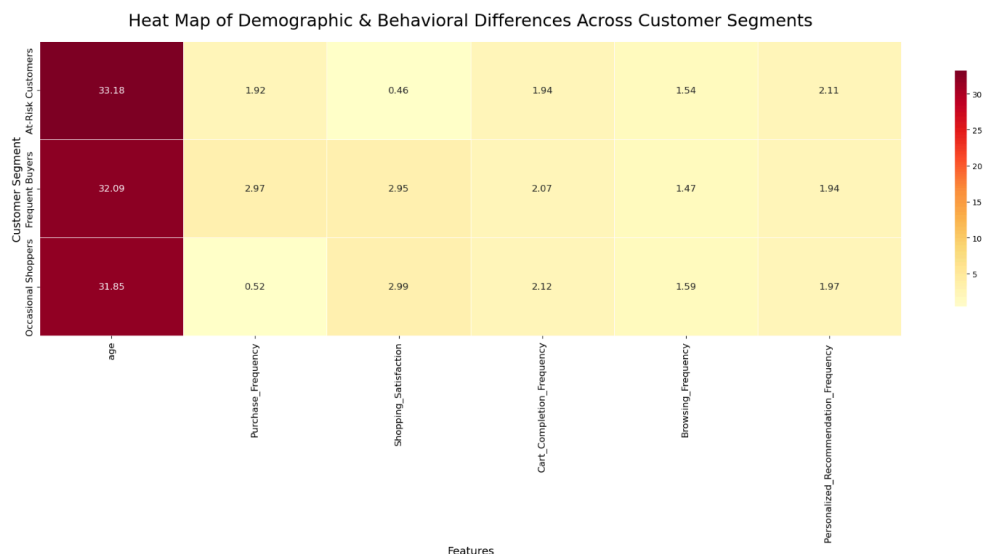
```
segment_summary
```

Customer_Profile	age	Purchase_Frequency	Shopping_Satisfaction	Cart_Completion_Frequency	Browsing_Frequency	Personalized_Recommendation_Frequency
At-Risk Customers	33.182336	1.917379	0.455840	1.940171	1.544160	2.113960
Frequent Buyers	32.092664	2.969112	2.953668	2.065637	1.474903	1.942085
Occasional Shoppers	31.847368	0.515789	2.989474	2.115789	1.594737	1.973684

Visualization heat map

```
plt.figure(figsize=(20, 10))
sns.heatmap(
    segment_summary,
    annot=True,
    fmt=".2f",
    cmap="YlOrRd",
    linewidths=0.5,
    cbar_kws={"shrink": 0.8},
    annot_kws={"size": 12}
)
plt.title(
    "Heat Map of Demographic & Behavioral Differences Across Customer Segments",
    fontsize=22,
    pad=20
)

plt.xlabel("Features", fontsize=14)
plt.ylabel("Customer Segment", fontsize=14)
plt.xticks(fontsize=12)
plt.yticks(fontsize=12)
plt.tight_layout()
plt.show()
```



Insights from the graph

Age is similar across segments , so demographics don't strongly differentiate groups.

Frequent Buyers show the highest purchase frequency and strong engagement.

Occasional Shoppers have higher shopping satisfaction but low purchase frequency.

At-Risk Customers have the lowest satisfaction and purchase frequency, indicating churn risk.

Cart completion, browsing, and recommendation usage are fairly similar across segments, with only minor differences.

Task 4: Recommendation and Review Insights

Examine the relationship between recommendation helpfulness and shopping satisfaction.

Correlation Analysis

```
correlation = df["Recommendation_Helpfulness"].corr(  
    df["Shopping_Satisfaction"]  
)
```

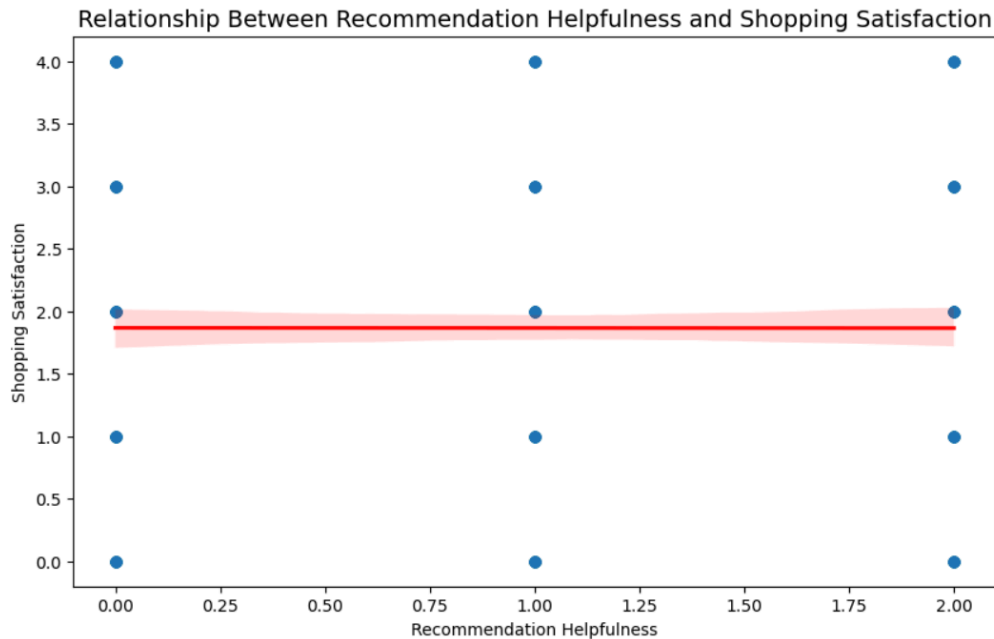
```
print("Correlation between Recommendation Helpfulness and Shopping  
Satisfaction:",  
      round(correlation, 3))
```

```
Correlation between Recommendation Helpfulness and Shopping Satisfaction: -0.001
```

Scatter Plot with Trend Line

```
plt.figure(figsize=(10, 6))  
sns.regplot(  
    x="Recommendation_Helpfulness",  
    y="Shopping_Satisfaction",  
    data=df,  
    scatter_kws={"alpha": 0.6},  
    line_kws={"color": "red"}  
)  
plt.title(  
    "Relationship Between Recommendation Helpfulness and Shopping Satisfaction",  
    fontsize=14  
)  
plt.xlabel("Recommendation Helpfulness")  
plt.ylabel("Shopping Satisfaction")
```

```
plt.show()
```



Insights from the graph

Very weak relationship: The nearly flat trend line shows little to no correlation between recommendation helpfulness and shopping satisfaction.

High variability: Satisfaction levels vary widely at each helpfulness level.

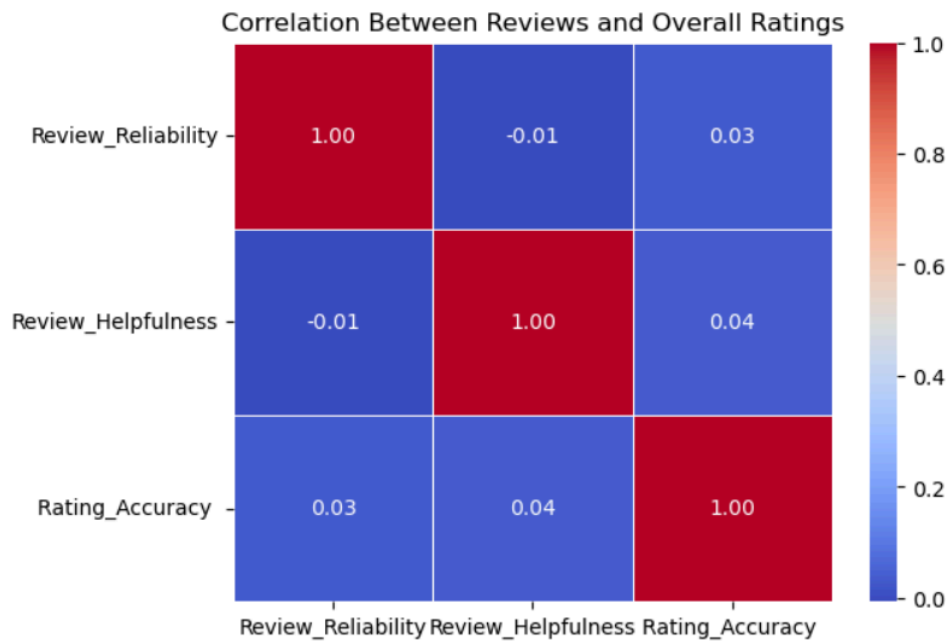
No clear upward trend: Higher recommendation helpfulness does not consistently lead to higher shopping satisfaction.

Evaluate how review reliability and helpfulness impact overall ratings.

```
selected = ["Review_Reliability", "Review_Helpfulness", "Rating_Accuracy"]
```

```
correlation = df_selected.corr()  
print("Correlation matrix:\n", correlation)
```

```
sns.heatmap(correlation, annot=True, cmap="coolwarm", fmt=".2f", linewidths=0.5)  
plt.title("Correlation Between Reviews and Overall Ratings")  
plt.show()
```



Insights from the graph

The graph suggests that Review Reliability, Review Helpfulness, and Rating Accuracy operate largely independently in this dataset.

If we are expecting these factors to reinforce each other, this result indicates:

Either the constructs are genuinely distinct, or

The relationship may be non-linear, context-dependent, or influenced by other variables not included here.

Identify trends in how often customers engage with or trust personalized recommendations.

Columns of interest

```
cols = ["Personalized_Recommendation_Frequency ",
"Recommendation_Helpfulness"]
```

Overall mean & median

```
overall_summary = df[cols].agg(["mean", "median"])
```

```
print("Overall Summary:\n", overall_summary)
```

Overall Summary:

	Personalized_Recommendation_Frequency	Recommendation_Helpfulness
mean	2.025	0.98625
median	2.000	1.00000

segment-wise

```
if "Customer_Profile" in df.columns:
```

```

segment_summary = df.groupby("Customer_Profile")[cols].agg(["mean",
"median"])
print("\nSegment-wise Summary:\n", segment_summary)

```

Segment-wise Summary:

Customer_Profile	Personalized_Recommendation_Frequency	
	mean	median
At-Risk Customers	2.113960	2.0
Frequent Buyers	1.942085	2.0
Occasional Shoppers	1.973684	2.0

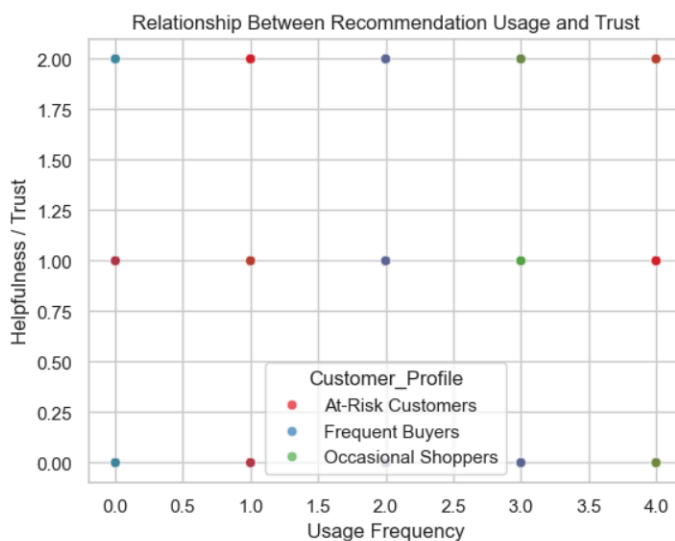
Customer_Profile	Recommendation_Helpfulness	
	mean	median
At-Risk Customers	0.985755	1.0
Frequent Buyers	0.934363	1.0
Occasional Shoppers	1.057895	1.0

visualization

```

sns.scatterplot(
    x="Personalized_Recommendation_Frequency ",
    y="Recommendation_Helpfulness",
    data=df,
    hue="Customer_Profile" if "Customer_Profile" in df.columns else None,
    palette="Set1",
    alpha=0.7
)
plt.title("Relationship Between Recommendation Usage and Trust")
plt.xlabel("Usage Frequency")
plt.ylabel("Helpfulness / Trust")
plt.show()

```



Suggest actionable insights for improving eBay's recommendation system.

Highlight personalized recommendations more prominently on homepage, search results, and product pages.

Use dynamic notifications or email prompts for users who rarely interact with recommendations.

Test different formats (carousel, lists, "Recommended for You") to see what boosts clicks.

Leverage purchase history, browsing behavior, and wishlists to improve recommendation relevance.

Implement feedback mechanisms, e.g., "Was this helpful?" to refine recommendation algorithms.

Use collaborative filtering combined with content-based filtering to surface products that similar users actually buy.

Prioritize highly rated and helpful products in recommendation lists.

Incorporate review scores, helpful votes, and verified purchases into recommendation algorithms.

Highlight why a product is recommended, e.g., "Based on your previous purchases" or "Top-rated by users like you."

Task 5: Visualization and Reporting

Create attractive visualizations (bar charts, heatmaps, pie charts) for:

Purchase categories

Browsing frequency distribution

Satisfaction levels

Correlation between recommendation usefulness and satisfaction

Summarize findings in a clear and visually appealing report format.

```
sns.set(style="whitegrid")
fig, axes = plt.subplots(2, 2, figsize=(18, 12))
sns.countplot(
    y="Purchase_Categories",
    data=df,
    order=df["Purchase_Categories"].value_counts().index,
    palette="Paired",
    ax=axes[0,0]
)
```

```

axes[0,0].set_title("Purchase Frequency by Category", fontsize=14)
axes[0,0].set_xlabel("Number of Purchases")
axes[0,0].set_ylabel("Purchase_Categories")

```

```

sns.histplot(df["Browsing_Frequency"], bins=15, kde=True, color="skyblue",
ax=axes[0,1])
axes[0,1].set_title("Browsing Frequency Distribution", fontsize=14)
axes[0,1].set_xlabel("Browsing Frequency")
axes[0,1].set_ylabel("Number of Customers")

```

```

sns.histplot(df["Shopping_Satisfaction"], bins=10, kde=True, color="lightgreen",
ax=axes[1,0])
axes[1,0].set_title("Shopping Satisfaction Levels", fontsize=14)
axes[1,0].set_xlabel("Satisfaction Level")
axes[1,0].set_ylabel("Number of Customers")

```

```

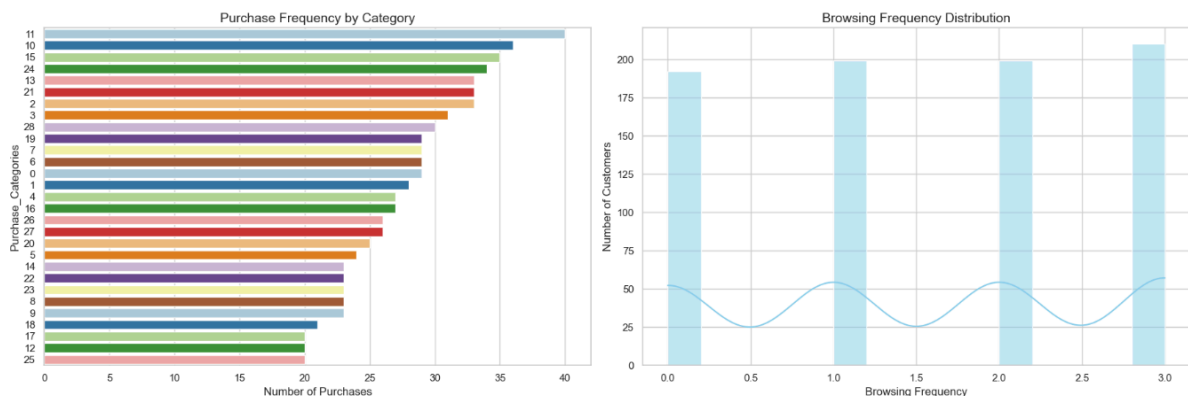
sns.scatterplot(
    x="Recommendation_Helpfulness",
    y="Shopping_Satisfaction",
    data=df,
    hue="Customer_Profile" if "Customer_Profile" in df.columns else None,
    palette="Set1",
    alpha=0.7,
    ax=axes[1,1]
)
axes[1,1].set_title("Recommendation Helpfulness vs Satisfaction", fontsize=14)
axes[1,1].set_xlabel("Recommendation Helpfulness")
axes[1,1].set_ylabel("Shopping Satisfaction")

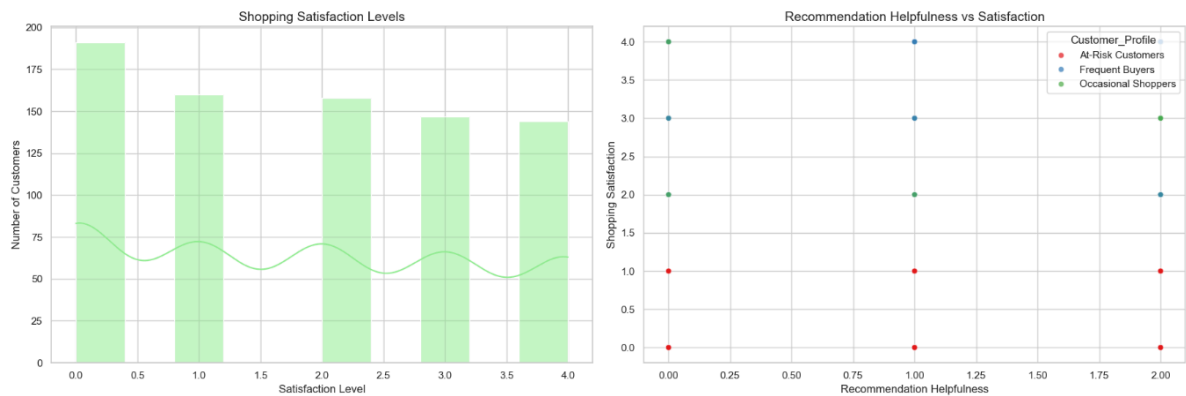
```

```

plt.tight_layout()
plt.show()

```





Insights from graph

- ## Optimize Recommendation Placement: Increase visibility for low-engagement users.
- ## Segment-Based Personalization: Frequent Buyers, Occasional Shoppers, At-Risk Customers.
- ## Focus on Trust & Helpfulness: Show highly rated, relevant items.
- ## Promote Underperforming Categories: Target marketing campaigns for less purchased products.
- ## Monitor Satisfaction: Track how recommendation changes impact overall satisfaction.

Highlight what drives customer transaction behavior and financial risk.

- ## Risk Factors
- ## Low satisfaction- High chance of churn or cart abandonment
- ## Infrequent purchasing- Unpredictable revenue streams
- ## Low recommendation trust- Missed upselling / cross-selling opportunities
- ## High cart abandonment - Lost potential transactions
- ## Limited category engagement- Over-reliance on few categories can affect revenue stability

Discuss data-backed recommendations for customer engagement or monitoring.

- ## Segment-Based Personalization: Tailor recommendations for Frequent Buyers, Occasional Shoppers, and At-Risk Customers.
- ## Optimize Recommendation System: Use purchase and browsing data to increase relevance; add feedback loops.
- ## Targeted Communication: Send personalized offers or reminders to low-engagement customers.
- ## Promote Underperforming Categories: Encourage exploration to balance revenue.

Track Key Metrics: Monitor purchase frequency, satisfaction, and recommendation trust.

Behavioral Analytics: Detect changes in browsing vs purchase patterns to identify at-risk users.

Financial Risk Alerts: Flag low satisfaction, high cart abandonment, or infrequent purchases for intervention.

Task 6: Video Presentation

[Video Link](#)